

User Management Report

Objective

This report summarizes the user directory, authorization model, effective role mappings, management scripts, and privilege propagation used by the libcloud platform, based on the attached `user_management.md` document.[file:4]

Introduction

The attached document describes a split design in which user identities live in LLDAP while authorization is enforced through OpenFGA tuples and derived relations.[file:4] Dex authenticates users against LLDAP and issues JWTs, while libcloud REST and OpenFGA use those identities as `user:<uid>` subjects for authorization decisions.[file:4]

Emphasis: The central design principle is that identity and authorization are intentionally separated. LLDAP stores who the user is, but OpenFGA determines what that user can actually do inside the platform.[file:4]

Identity Layer

All end-user identities are stored in LLDAP, which is deployed in a sibling compose project at `../lldap`.[file:4] OpenFGA does not store user records directly; it only stores relationship tuples that reference LLDAP users through `user:<uid>` subjects.[file:4]

Users are created by `setup.sh` through the `lldap-tools` container using `lldap_ensure_user.sh`, while the built-in admin account is reserved for LLDAP administration and is not treated as a libcloud principal.[file:4]

UID	Display name	Email	Origin
admin	Administrator	—	LLDAP directory admin, not a libcloud principal.[file:4]
superadmin	Super Admin	superadmin@libcloud.local	Platform bootstrap user from <code>setup.sh</code> .[file:4]
aws-owner	AWS Owner	aws-owner@libcloud.local	AWS tenant owner user.[file:4]
aws-admin	AWS Admin	aws-admin@libcloud.local	AWS tenant admin user.[file:4]
aws-viewer	AWS Viewer	aws-viewer@libcloud.local	AWS tenant viewer user.[file:4]
ntnx-owner	Nutanix Owner	ntnx-owner@libcloud.local	Nutanix tenant owner user.[file:4]
ntnx-admin	Nutanix Admin	ntnx-admin@libcloud.local	Nutanix tenant admin user.[file:4]

UID	Display name	Email	Origin
ntnx-viewer	Nutanix Viewer	ntnx-viewer@libcloud.local	Nutanix tenant viewer user.[file:4]
cloud-denied	Cloud Denied	cloud-denied@libcloud.local	Denial-demo user with no effective OpenFGA role.[file:4]

Management Operations

The document defines separate management toolsets for LLDAP and OpenFGA, with both controlled through scripts and gated by administrative credentials or a valid superadmin JWT. [file:4] LLDAP operations depend on LLDAP_ADMIN_PASS, while OpenFGA operations require SUPERADMIN_JWT obtained from scripts/superadmin_auth.sh.[file:4]

Layer	Operation examples	Scripts
LLDAP	Add, delete, reset password, list groups, audit memberships, manage groups.[file:4]	lldap-user-onboard.sh, lldap-user-offboard.sh, lldap-user-password-reset.sh, lldap-audit-all-memberships.sh, group scripts.[file:4]
OpenFGA	Write/delete tuples, check access, list objects/users, audit tuples, reconcile model state.[file:4]	openfga-tuple-write.sh, openfga-tuple-delete.sh, openfga-check.sh, openfga-list-objects.sh, openfga-tuple-audit.py, openfga-tuple-reconcile.py.[file:4]

A notable operational detail is that the live OpenFGA store contains exactly the 17 tuples seeded by openfga_bootstrap.py::INITIAL_TUPLES, with no legacy or orphan tuples in this version of the document.[file:4]

Authorization Model

The active OpenFGA model is defined in openfga_bootstrap.py::LIBCLOUD_MODEL and uses schema 1.1, with store metadata written to generated/fga.env after bootstrap.[file:4] The active object types are limited to user, platform, tenant, libcloud_api, provider, aws_region, and nutanix_cluster; the document explicitly states that there are no legacy role or api_scope object types in the active model.[file:4]

Object type	Key relations
user	Terminal subject type; no relations.[file:4]
platform	superadmin, can_manage_platform.[file:4]
tenant	owner, admin, viewer, member, can_assign_*, can_manage_credentials, can_provision, can_read.[file:4]
libcloud_api	parent, can_connect.[file:4]
provider	parent, allowed, can_use.[file:4]
aws_region	provider, tenant, operator, viewer, tenant_owner, tenant_admin, tenant_viewer, can_provision, can_read.[file:4]

Object type	Key relations
nutanix_cluster	Same relation set as aws_region.[file:4]

The only role-bearing relations directly assigned to users are superadmin on platform:main and owner / admin / viewer on tenant:aws and tenant:nutanix.[file:4] Other permissions such as can_connect, can_use, can_provision, and can_read are derived by the model rather than granted directly to users.[file:4]

Emphasis: This model is intentionally compact. Direct assignment is restricted to a small number of human-understandable roles, while operational permissions are computed from those roles, which improves consistency and reduces policy drift.[file:4]

Effective Role Mapping

The effective role map is fully defined by the seeded tuples in INITIAL_TUPLES, comprising platform-level superadmin access, break-glass tenant ownership for superadmin, and per-tenant owner/admin/viewer assignments for AWS and Nutanix.[file:4] The cloud-denied user authenticates through Dex but has no OpenFGA subject tuple, so it is denied at can_connect by design.[file:4]

LLDAP UID	OpenFGA subject	Relation	Object	Effective role
superadmin	user:superadmin	superadmin	platform:main	Platform superadmin.[file:4]
superadmin	user:superadmin	owner	tenant:aws	Break-glass owner for AWS tenant.[file:4]
superadmin	user:superadmin	owner	tenant:nutanix	Break-glass owner for Nutanix tenant.[file:4]
aws-owner	user:aws-owner	owner	tenant:aws	AWS owner.[file:4]
aws-admin	user:aws-admin	admin	tenant:aws	AWS admin.[file:4]
aws-viewer	user:aws-viewer	viewer	tenant:aws	AWS viewer.[file:4]
ntnx-owner	user:ntnx-owner	owner	tenant:nutanix	Nutanix owner.[file:4]
ntnx-admin	user:ntnx-admin	admin	tenant:nutanix	Nutanix admin.[file:4]
ntnx-viewer	user:ntnx-viewer	viewer	tenant:nutanix	Nutanix viewer.[file:4]
cloud-denied	(none)	—	—	Authenticated but denied at can_connect.[file:4]
admin	(none)	—	—	Directory admin only, not a libcloud principal.[file:4]

Structural Grants and Privileges

The model relies on non-user structural tuples to connect tenants to the API, providers, and backend resources, allowing tenant roles to propagate into actual runtime permissions.[file:4] These tuples are not user roles, but they are essential because they form the graph that makes derived checks like `can_connect`, `can_use`, `can_read`, and `can_provision` work.[file:4]

Subject	Relation	Object	Purpose
tenant:aws	parent	libcloud_api:main	AWS members can connect to the API.[file:4]
tenant:nutanix	parent	libcloud_api:main	Nutanix members can connect to the API.[file:4]
tenant:aws	parent	provider:aws	AWS members can use AWS provider path.[file:4]
tenant:nutanix	parent	provider:nutanix	Nutanix members can use Nutanix provider path.[file:4]
provider:aws	provider	aws_region:aws	AWS backend link.[file:4]
tenant:aws	tenant	aws_region:aws	AWS role propagation path.[file:4]
provider:nutanix	provider	nutanix_cluster:nutanix	Nutanix backend link.[file:4]
tenant:nutanix	tenant	nutanix_cluster:nutanix	Nutanix role propagation path.[file:4]

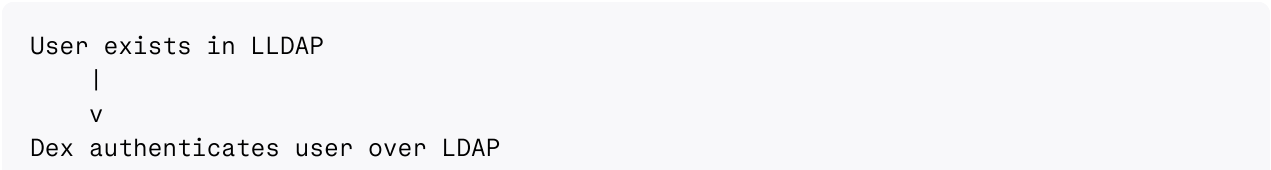
Privilege behavior is summarized as follows:[file:4]

Role	Main capabilities
superadmin	Platform bootstrap identity; gates Vault seeding, OpenFGA tuple/model changes, and LLDP user CRUD; also break-glass owner on both tenants.[file:4]
owner	Can assign owner/admin/viewer, manage backend credentials, provision, and read.[file:4]
admin	Can assign viewer, provision, and read, but cannot assign admin/owner or manage credentials.[file:4]
viewer	Read-only and enumeration access; cannot provision or assign roles.[file:4]
cloud-denied	Authenticates but fails <code>can_connect</code> because no OpenFGA role exists.[file:4]

The document also states that cross-cloud isolation is enforced: AWS tenant users have no relation on Nutanix provider or backend objects, and Nutanix tenant users have no relation on AWS provider or backend objects.[file:4]

Flowchart

The following flow summarizes how a user becomes effective inside the platform.[file:4]



```
|
v
JWT is issued to libcloud components
|
v
OpenFGA resolves subject as user:<uid>
|
v
Direct role tuple exists?
|
+--> No -> user authenticates but fails can_connect / access checks
|
v
Role attaches to tenant or platform
|
v
Structural tuples propagate access to API, provider, and backend objects
|
v
Derived permissions become effective
|
v
User receives allowed or denied access based on can_read / can_provision / can_manage
```

Conclusion

The attached user-management design is a clean separation of concerns in which LLDAP acts as the identity source, Dex handles authentication, and OpenFGA determines effective authorization through a small, explicit set of seeded tuples and derived relations.[file:4] The model is concise, current-state focused, and easier to audit than earlier variants because this version explicitly contains only active object types and exactly 17 seeded tuples with no legacy orphan state.[file:4]

From an operational perspective, the most important checks are whether a user exists in LLDAP, whether that user has an active role-bearing tuple in OpenFGA, and whether the structural tuples connect the tenant to the relevant API, provider, and backend objects.[file:4] This makes the platform predictable to reason about, enforces cross-cloud isolation between AWS and Nutanix tenants, and supports least-privilege role separation through derived permissions rather than broad direct grants.[file:4]